



Large language models based vulnerability detection: How does it enhance performance?

Cho Do Xuan¹ · Dat Bui Quang² · Vinh Dang Quang¹

Accepted: 7 January 2025 / Published online: 24 January 2025

© The Author(s), under exclusive licence to Springer-Verlag GmbH Germany, part of Springer Nature 2025

Abstract

Detecting vulnerabilities in C/C++ source code has become a critical challenge in information security, especially as the growing number and severity of new vulnerabilities increasingly impact organizations. In this context, Large Language Models (LLMs) have emerged as a promising approach; however, building a model capable of effectively predicting and classifying various types of vulnerabilities from diverse datasets remains a complex problem, demanding innovative and comprehensive solutions. Our research proposes a breakthrough approach by developing the FG-CVD ensemble learning model, an advanced architecture that combines code embedding techniques and Knowledge Argument to enhance feature representation and the ability to learn complex relationships within source code. These improvements are specifically designed on the foundation of code embedding and the Transformer architecture of LLMs to boost the detection and classification of sophisticated vulnerability patterns. To evaluate the model's effectiveness, we conducted extensive experiments on four representative datasets: Reveal, BigVul, RealVul, and FFMQ + QEmu. The experimental results demonstrated FG-CVD's superior performance with an average accuracy of 85%, a prediction precision of 43%, a recall of 65%, and an F1-score of 47%. Notably, the model exhibited flexible adaptability to datasets with different structures and efficiently addressed data imbalance between labels. Moreover, through rigorous cross-dataset testing, the model showcased strong generalization capabilities and high stability, underscoring not only the academic value of the approach but also its practical potential, outperforming traditional approaches across a range of metrics and experimental scenarios.

Keywords Software vulnerability detection · Embedding code · Transformer encoder · Large language models · Ensemble learning

1 Introduction

1.1 Overview of source code vulnerability detection

Software security vulnerabilities represent significant potential threats to modern computing systems [1, 3, 5, 37]. Source code vulnerabilities constitute a primary security risk in software systems. In practice, the successful use and exploitation of source code vulnerabilities can lead to severe consequences for organizations and agencies,

including data breaches, financial losses, and operational disruptions. According to statistics in [2, 3, 25], source code vulnerabilities have caused substantial damage to organizations' assets and reputation [39]. Furthermore, as reported in [25, 50], the number of discovered and disclosed source code vulnerabilities is increasing exponentially. This necessitates the development of advanced and sophisticated source code vulnerability detection techniques to replace traditional approaches.

Studies [3, 25] have identified two primary methods for detecting source code vulnerabilities. The first method relies on pattern matching against known Common Vulnerabilities and Exposures (CVEs) and Common Weakness Enumerations (CWEs) databases. However, this approach has limitations, particularly in its inability as it cannot detect zero-day vulnerabilities. The second approach, which has gained significant attention in recent years, leverages

✉ Cho Do Xuan
chodx@ptit.edu.vn

¹ Department of Information Security, Posts and Telecommunications Institute of Technology, Hanoi, Vietnam

² Department of Information Technology, Posts and Telecommunications Institute of Technology, Hanoi, Vietnam

anomaly detection techniques combined with artificial intelligence methodologies for more sophisticated vulnerability identification. The second approach, which has gained significant attention in recent years, leverages anomaly detection techniques combined with artificial intelligence methodologies for more sophisticated vulnerability identification.

In the context of vulnerability detection, feature analysis techniques, recent studies predominantly employ two main approaches: Graphs and Natural Language Processing (NLP). Graph-based approaches typically represent source code in several main forms such as Abstract Syntax Tree (AST) [4, 5, 13], Code Property Graph (CPG) [14, 42, 52–54, 57], Control Flow Graph (CFG) [25], Program Dependence Graph (PDG) [22] and then use Deep Graph Neural Networks (DGNNs) to extract features from these graphs.

In contrast, for NLP methods, code embedding techniques are considered a primary choice. Some NLP models and LLMs currently of interest in this area include Word2vec [56], Roberta [12, 55], CodeT5 [7–9], CodeBert [10, 11],...

1.2 Assessment of trends in source code vulnerability detection using graph analysis techniques

Significant research efforts in graph-based source code vulnerability detection, have been documented in several seminal works [45–47]. Specifically, these studies employ a common methodology is to transform source code into graph representations, such as AST, CFG, CPG, and PDG, which are then processed by Deep Neural Networks (DNNs) to extract meaningful features.

Through comprehensive analysis and evaluation of these graph-based approaches for source code vulnerability detection, we have identified several key issues:

First, approaches using AST [21], PDG [22], and CFG [23] have certain limitations, and to address these shortcomings, CPG [14, 19, 20, 49] is an optimal and favorable choice. This is because CPG includes all three components: AST, PDG, and CFG. Although CPG effectively represents the characteristics of the source code [14, 19, 20], using CPG to represent large code segments can lead to a CPG that is oversized and cumbersome [35, 47].

Second, even with CPG, the graph-based approach often overlooks significant program features. Specifically, research [27] has indicated that using graphs focuses only on structural features and ignores the semantic features of the program. This not only causes noise and redundant information but also affects the speed and computation time when extracting source code. To solve this problem, some recent proposals have introduced techniques for trimming graphs in general and CPGs in particular [35, 47]. However, the effectiveness of these methods is not high and is challenging to apply in

practice because the trimming process can result in the loss of critical information due to the inability to identify the factors causing source code vulnerabilities.

1.3 Evaluation of the source code vulnerability detection approach based on LLMs

Recent studies suggest that current LLMs provide robust capabilities for analyzing and understanding source code [16]. Specifically, studies [6, 15, 16] have shown that LLMs with modern architectures offer greater effectiveness for source code feature extraction compared to traditional NLP techniques. The reason for this is that LLMs are built based on Transformers. These models are trained on unlabeled data related to source code, which allows LLMs to create embedding vectors in various programming languages [7, 10, 12], thereby providing the ability to handle diverse source code. Specifically, some pre-trained models, such as CodeBERT [10], CodeT5 [9], and Roberta [12, 55], using transformer-based training mechanisms, are effective in understanding the structural aspects of source code.

Despite these outstanding advantages, LLMs still have several issues that need improvement:

First: Limited capacity in processing extensive and complex code segments. Studies [18] have shown that the size of embedding models in LLMs remains insufficient to fully address the complexity of real-world source code. Specifically, RoBERTa uses the Transformer Encoder architecture, similar to BERT, but with adjustments based on global attention mechanisms and encoder layers for text processing. Consequently, this architecture renders the RoBERTa model computationally complex and resource-intensive, which increases training and inference costs [12], and makes it challenging to capture long contexts and specific code structures.

In a parallel architectural approach, CodeBERT implements the Transformer Encoder architecture similar to BERT but distinguishes itself through specialized training with source code data. Although CodeBERT is optimized for source code, it exhibits limitations in processing long, complex, or less common code segments, often requiring truncation of code snippets or resulting in the loss of important semantic information, which reduces model effectiveness. Furthermore, in the context of cross-platform compatibility, UniXCoder [31] does not generalize well when switching between different contexts or programming languages, reducing its effectiveness in multi-platform applications.

Second: The performance of LLMs: In a study [32], there have been doubts about the performance of LLMs. The authors questioned: “Do LLMs really learn the syntax and semantics of code from training data and leverage this knowledge to perform source code detection tasks?”. Similarly, a study [17] highlighted a disadvantage of current LLM

solutions: They do not maintain robustness when faced with adversarial attack techniques.

1.4 Motivation and objectives

1.4.1 Motivation

Based on the analysis and evaluation of the strengths and weaknesses of the two approaches, namely graph-based analysis and LLMs, we have identified that applying LLMs for the task of source code vulnerability detection is one of the most promising approaches, especially with large datasets. However, to use LLMs optimally and efficiently, certain improvements are needed. Specifically, new approaches and proposals based on LLMs need to address the two major issues faced by traditional models.

First: How to handle long and complex code segments without losing important information about the structure and semantics of the source code. This challenge is common among encoder and transformer models within LLMs.

Second: How to ensure that the efficiency of new models does not depend on certain pre-trained datasets. This is because, as noted in the study [43], approaches based on anomaly feature analysis and artificial intelligence often perform well on specific datasets. As a result, even though studies yield good results during training and testing, they may not perform well on other datasets. This issue arises because the proposed data extraction and representation methods are tailored to the chosen datasets, lacking generalization [18, 43].

1.4.2 Objectives

To tackle these two issues, we consider it essential to develop a new model leveraging the concept of Ensemble Learning. The essence of this method is to combine different solutions and models to create a more complete solution that leverages the strengths of each model. In the field of information security, some recent approaches using Ensemble Learning for network attack detection have shown clear effectiveness [18, 19, 22, 40]. For the task of source code vulnerability detection, notable studies using Ensemble Learning include [24–26, 36, 45]. These studies have utilized the strengths of individual models to form a more robust and effective model for detecting source code vulnerabilities. This motivation has led us to propose a new model called FG-CVD. This proposed model will be a combination of three main components: embedding coder, Knowledge Argument (KA), and Multi-layer Perceptron (MLP). Specifically:

Embedding coder technique: This component embeds source code into a unique feature vector. To handle long and complexly structured source code segments, this paper proposes a new embedding coder model that includes the follow-

ing processing methods and techniques: Bidirectional Self-Attention, Feed Forward, and Projection Layer. The details of the operation and computational techniques of these blocks will be presented in Section 3.2 of the paper.

KA technique: This is an information synthesis technique. We have observed that the source code, once embedded into a vector, contains comprehensive information about context and important features. However, this is not sufficient for the vector to represent long and complex source code. To overcome this issue, we propose using an additional KA layer right after the Encoding code model. The KA layer is built on the idea of the Transformer Encoder to enhance the knowledge of features within the source code. To achieve the set goal, the KA layer will include several key processing techniques: MLP, Feedforward, and Add&Norm. The operational process and computational techniques of the components in the KA layer will be presented in Section 3.3 of the paper.

MLP network: It functions to classify feature vectors of source code to determine which source codes are normal and which contain security vulnerabilities.

The main contributions of this work are summarized as follows:

- The proposed FG-CVD model, based on the Ensemble Learning method, has achieved two major objectives: First, enhancing the effectiveness of source code vulnerability detection; second, improving the adaptability and stability of the model when applied in real-world scenarios.
- Propose an encoding model using the fine-tuning method of LLMs.
- Propose a KA model architecture based on the traditional Transformer model architecture.

2 Related studies

2.1 LLM-based source code vulnerability detection

Studies [6, 16, 38] have demonstrated the robustness of LLMs by significantly increasing the number of tokens and parameters, thereby improving the understanding and representation of code. Specifically, CodeBERT [10] is pre-trained using CodeSearchNet, which contains 2.3 million functions from six different programming languages: (Go, Java, JavaScript, PHP, Python, and Ruby). This model performs pre-training with Masked Language Modeling (MLM) and replaced token detection. During pre-training, each input is a pair of natural language descriptions and source code, where the text describes the meaning of the code. The MLM training in CodeBERT ensures that tokens from both the natural language and source code sections are masked, and the replaced token detection disrupts both parts of the input. Meanwhile,

Table 1 Some approaches using LLMs for source code vulnerability detection

Model	Model size	Embedding dimension	Max sequence length	Training data source	Date
CodeBERT [10]	125 M	768	512	CodeSearch Net	2020–2
UnixCoder [31]	125 M	768	1024	CodeSearch Net	2022–3
OpenAI-Embedding-Ada-002 [51]	350 M	1536	8191	Unknown	2022–12
CodeT5 + [33]	110 M	768	512	CodeSearch Net and GitHub	2023–5
CodeSage [51]	356 M	1024	1024	CodeSearch Net	2024–2

the UnixCoder model [31] is pre-trained using three types of language modeling tasks: Masked Language Modeling, Unidirectional Language Modeling, and Denoising Objective, which allows the model to support different types of downstream tasks. Furthermore, to improve the semantics of code segments, the model applies: multimodal contrastive learning leveraging AST to enhance the semantics of code embeddings and multimodal creation using code comments to align embeddings across programming languages. Study [9] proposed the CodeT5 model using the same basic transformer architecture as T5. CodeT5 is pre-trained on both CodeSearchNet data and additional C/C# projects from GitHub. Besides the objective of predicting masked spans, CodeT5 uses knowledge about whether tokens are identifiers (variable names or function names) and designs two new pre-training tasks. The new pre-training tasks are predicting masked identifiers (masking all identifiers) and identifier tagging (predicting whether a token is an identifier). To address some of the issues faced by studies using LLMs in handling large amounts of data, study [51] introduced the CodeSage model based on a combination of RiMLM, contrastive learning with hard negatives and hard positives. Experimental results showed that CodeSage [51] outperformed other models such as Code2code search and NL2code. However, CodeSage still faces challenges in error analysis and source code vulnerability detection compared to other models. Table 1 below lists some LLMs applied in the task of source code vulnerability detection. It provides an overview of key specifications, including model size, embedding dimensions, maximum sequence length, training data sources, origins, and release dates. These approaches are compared based on critical factors to assess their effectiveness and applicability in software security. This comparison highlights the evolution of LLM capabilities and their potential for addressing the challenges in vulnerability detection.

2.2 Source code vulnerability detection using ensemble learning methods

In the study [20], Cho proposed the GRD model for detecting source code vulnerabilities based on the CPG analysis

technique. By combining deep graph networks and NLP techniques, it synthesizes and extracts features of the source code in the form of a CPG attribute graph. The Dropout method is then used to balance the data for labels. However, these methods do not consider heterogeneous relationships in the source code structure graph. Heterogeneous relationships are different types of edges connecting different types of nodes in the graph, which leads to less effective graph representation learning. Moreover, these methods are limited in capturing long-range dependencies due to the depth levels in the source code structure graph. The study [27] proposed a model combining heterogeneous structural information with MetaPaths, a pre-trained large language model, and a graph self-attention encoder (GRSA). Similarly, according to the approach of [27, 45] proposed a MAGNET model learning Attentional Graph based on multi-detail Meta-paths for each code snippet. Along with a hierarchical graph neural network based on meta-paths to capture the relationships between distant nodes in the graph was also proposed.

With a similar approach, the study [15] proposed the GRACE model using the integration of AST, CFG, PDG, and LLMs. The authors argue that using LLMs will allow capturing more structural information of the code, creating a new basic prompt design consisting of domain information and identification information, as well as adding graph structures prompt and in-context learning prompt. In the experimental section, based on the evaluation results, the model improved Recall and F1-score metrics. It can be seen that this method works well in helping LLMs understand the structure of the code but lacks an attention structure that can capture vulnerabilities. Additionally, the architecture of this model is only suitable for short code snippets. The authors focused on improving the code structure understanding for LLMs, resulting in a very large prompt size causing high noise, which reduces the effectiveness of capturing vulnerability-related information. The study [23] also introduced the CSGVD neural network model for vulnerability detection at the function level. Accordingly, the study seeks to represent the source code in the form of CFG, then uses the embedding weight layer from a pre-trained language model to initialize the embedding layer for CSGVD. Finally, BiLSTM is used to

Table 2 Some Ensemble Learning Models

Approach	Feature extraction method	Data balancing method	Classification method
GRD [20] Roberta for source code	GCN for CPG	Dropout	softmax
AMPLE [48]	GCN and CNN	No	softmax
MAGNET [45]	GNN and Attention	No	cross-entropy
LIVABLE [46]	GNN and Bi-LSTM	No	Adaptive re-weighting
SySeVR [47]	word2vec and BGRU	No	Sigmoid

aggregate the local semantic information of the nodes and then based on M-BFA. The model achieved superior detection capability and reached an accuracy of 64.46% on the SARD dataset. However, the scarcity of high-quality data makes vulnerability detection models insufficient in capturing vulnerability patterns. The studies [5, 26] have proposed solutions to address this issue. However, the results are still not sufficient to provide good effectiveness in real-world vulnerability detection. Table 2 below lists and describes some ensemble learning models for source code vulnerability detection. The table outlines key aspects such as feature extraction methods, data balancing techniques, and classification methods. These models demonstrate varied strategies to enhance detection performance and adaptability in handling software vulnerabilities.

Based on the analysis and evaluation of Ensemble Learning models, as well as the assessment of some limitations faced by approaches using single LLMs [15], in this paper, we propose the FG-CVD model. This model differs from approaches using LLMs in 2 ways: First, it uses a new code encoding set based on improvements of traditional code encoding models in LLMs. Second, it employs a new KA information synthesis technique based on the transformer model. In the experimental section of the paper, we will conduct experiments to demonstrate that FG-CVD is more effective compared to single LLM models as well as other approaches.

3 Proposed FG-CVD model

3.1 FG-CVD model architecture

Figure 1 illustrates the architecture of the proposed FG-CVD model.

Figure 1 illustrates the operating principles of the proposed model, as follows: First, similar to the methods used in studies [20, 42, 45], the source code files are preprocessed to clean redundant or noisy data, extra whitespace, comments, etc., through the "preprocessing" block. Then, the cleaned source code snippets from the "preprocessing" block are passed through the Embedding Code block to convert from text data to feature vectors. Next, these feature vectors are passed through the KA layer for synthesis, normalization, and extraction of features to enrich information about the abnormal behavior of the source code. Finally, supervised machine learning algorithms in the "Classification" block will predict and detect normal source code files and files containing vulnerabilities. Algorithm 1 below details the operational process of the FG-CVD model

Algorithm 1 Source code vulnerability detection method based on the FG-CVD model

Input: Source Code
Output: Model Trained

Function Train(*SrcCode*):

```

PreProc ← Preprocess(SrcCode)
for i ← 1 to  $N_x$  do
    AttnOut ← BiSelfAttn(PreProc)
    PreProc ← FeedForward(AttnOut)

EmbCode ← ProjLayer(FeedForwardOut)
EmbCode ← PosEnc(EmbCode)
for i ← 1 to  $N_x$  do
    MLPOut ← MLP(EmbCode)
    AddNormOut ← AddNorm(MLPOut)
    FFOut ← FeedForward(AddNormOut)
    EmbCode ← AddNorm(FFOut)
ClassOut ← Classify(EmbCode)
return ClassOut

```

3.2 Source code embedding method based on the FG-CVD model

3.2.1 Building the embedding code block for FG-CVD

Based on the analysis and evaluation of the advantages and disadvantages of traditional Transformer Encoder models, this paper proposes a new Code Embedding model that inherits and enhances the strengths of existing Transformer Encoders. From Figure 1, it can be seen that the Code Embedding technique in the FirGen-CVD model will include several computational and processing methods: Bidirectional Self Attention, Feed Forward, and Projection Layer.

The Bidirectional Self Attention block highlights the important features of each token. The processing is represented by the following formulas:

$$Q = E * W_q + b_q \quad (1)$$

$$K = E * W_k + b_k \quad (2)$$

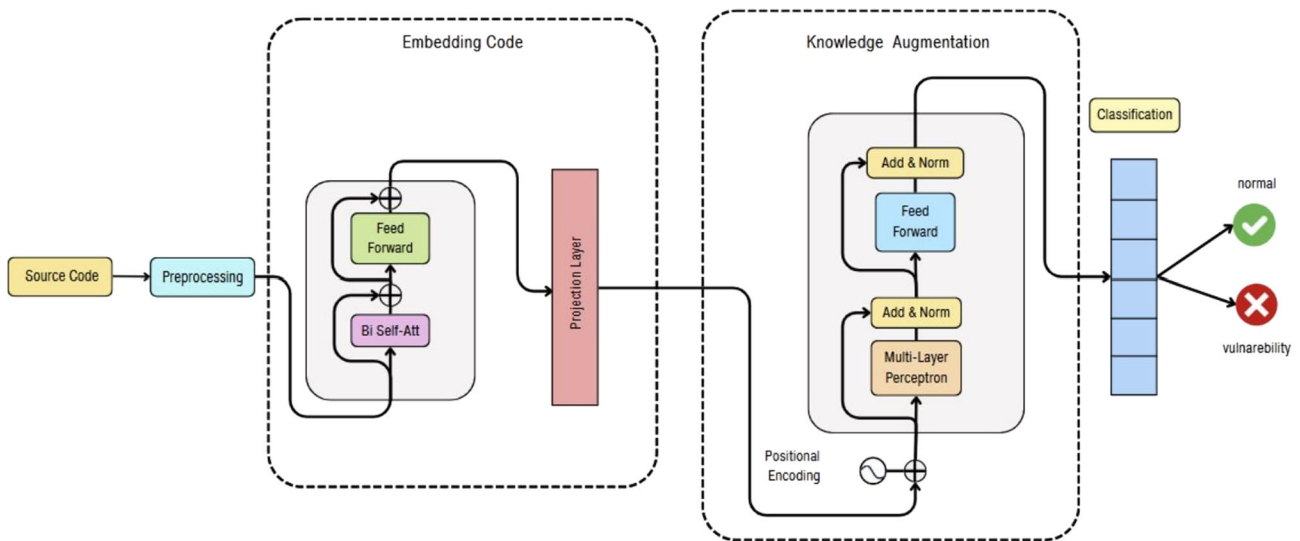


Fig. 1 The Embedding Code and Knowledge Augmentation process of the FG-CVD model in the task of source code vulnerability detection

$$V = E * W_v + b_v \quad (3)$$

where:

$W_q, W_k, W_v \in R^{d \times n}$ are the weight matrices of the linear layer and b_q, b_k, b_v are the bias terms.

n is the dimension is defined by the following formula: $n = \lfloor \frac{d}{num} \rfloor$, where num is the number of Attention heads.

$$S = \frac{Q * K^T}{\sqrt{n}} = [s_{ij}] \in R^{k \times k} \quad (4)$$

$$B = [b_{ij}] \in R^{k \times n}, b_{ij} = \frac{\exp(s_{ij})}{\sum_j \exp(s_{ij})} \quad (5)$$

$$A = \text{Concat}(B * V)_{num} * W + b \quad (6)$$

where:

$W \in R^{n \times n}$ and $b \in R^{k \times n}$ are the weight matrix and bias of the linear layer.

The Feed Forward and Projection Layer blocks enrich the information of the important features in each token and also reduce the dimensions of the tokens to improve computation speed. The computational and processing formulas inside these blocks are represented as follows:

$$R = \text{ReLU}((A + E) * W_1), W_1 \in R^{d \times p} \quad (7)$$

$$M = R * W_2, W_2 \in R^{p \times d} \quad (8)$$

$$Z = M + A + X, Z \in R^{n \times d} \quad (9)$$

Matrix Z is a matrix containing the embedding information of N tokens. This matrix can be very large if we are

processing long and complex source code. To address this issue, we will only use the first token [CLS], which is the first row of matrix Z . The processing is represented by the following formulas:

$$z_1 = I * Z, I \in R^{1 \times n} \quad (10)$$

Vector $\mathbf{I} = [1, 0, \dots, 0]$ serves to extract the first token. The [CLS] token, acting as an aggregate token, receives features accumulated from other tokens through the Bidirectional Self Attention block. Once this aggregation process is complete, the [CLS] token becomes a representation of the entire source code sequence and encapsulates important information from all other tokens. To optimize representational capacity and improve model accuracy, we will further pass this [CLS] token through a Projection Layer. This layer not only changes the size of the token but also enhances the information within. The Projection Layer amplifies important synthesized features, filters out unnecessary information, and highlights the data relevant to the main task, in this case, classification or detecting security vulnerabilities in source code. Using the Projection Layer is an essential step to ensure that the [CLS] token not only carries features of the code sequence but also contains the most critical information for the model's decision-making process. Through this layer, crucial features are amplified, enabling the model to more easily identify code patterns related to vulnerabilities or weaknesses in the source code, while reducing unnecessary excess information. As a result, the [CLS] token becomes more information-rich after passing through the Projection Layer, ready for subsequent processing steps in the model network:

$$T = z_1 * W_t, W_t \in R^{D \times D} \quad (11)$$

Finally, the normalization of matrix T is calculated using the formula:

$$T := \frac{T}{\max(\|T\|_2, \varepsilon)} \quad (12)$$

Here, we set $\varepsilon = 10^{-12}$ to avoid division by zero. Matrix T is normalized using the L_2 norm to facilitate subsequent processing steps.

3.2.1.1 Source code embedding process based on FG-CVD

Based on the description in Figure 1 and the processing formulas within each block, the operation process of the Code Embedding technique in the FG-CVD model includes the following steps: First, the original source code is split into K tokens using SentencePiece and Byte-Pair Encoding algorithms. Each token is then embedded into a D -dimensional vector using an embedding matrix, where the matrix has as many rows as the number of tokens in the vocabulary, with each row being the embedding vector of the token at that position. The mathematical representation is as follows:

$$SourceCode \rightarrow T_i \in R^{k \times 1} \rightarrow E \in R^{k \times d} \quad (13)$$

where:

T_i is a vector containing the indices of the tokens in the vocabulary.

E is the embedding matrix that maps each token index to a vector.

We pass these K tokens through the Bidirectional Self Attention block to enhance the relationships between tokens. Tokens that are closely related will have higher values. The model then calculates the attention weights for each token pair, with more closely related pairs receiving higher attention weights. After calculating the attention weights, the data is further processed through a linear layer to enrich the information about related tokens. Subsequently, after the source code has been processed by the Bidirectional Self Attention block, it is passed to the FeedForward layer. Here, the FeedForward layer further processes the data through two layers. The output of this block will be a matrix with rows equal to the number of tokens and columns equal to the embedding dimension. Finally, the first token is selected to be passed to the Projection Layer. This layer reduces the data dimensions for computational efficiency and normalizes the data according to the L_2 norm for easier information processing. This process is continued through N_x layers to enrich the token information.

3.3 Building the KA block for FG-CVD

3.3.1 Scientific basis for proposing KA

Our KA architecture is an adaptation from traditional Transformer Encoders, aimed at processing input data which are individual vectors. When processing multiple source codes in parallel, the model's input becomes a matrix where each row represents a source code, and each column represents the embedding dimensions of that source code. However, applying traditional Transformer Encoders in this case may lead to the loss of important information because the Multi-head Attention mechanism tends to link source codes together without considering whether they come from the same distribution. To clarify this issue, we used the Maximum Likelihood Estimation (MLE) method to estimate the distribution of the input source codes. Suppose that the source codes $X_1, X_2, \dots, X_n$ are independent random variables following a distribution $p(X_i|\theta)$ with parameter θ , we estimate the parameter θ to maximize the likelihood of the observed data.

$$L(\theta) = p(X_1, X_2, \dots, X_n|\theta) = \prod_{i=1}^n p(X_i|\theta) \quad (14)$$

$$l(\theta) = \log L(\theta) = \sum_{i=1}^n \log p(X_i|\theta) \quad (15)$$

$$\frac{\partial l(\theta)}{\partial \theta} = 0 \quad (16)$$

If the estimated θ indicates that the source codes $X_1, X_2, \dots, X_n$ all come from the same distribution, then using the Attention mechanism might be justified. However, if the source codes come from different distributions, linking them through Attention could introduce noise, leading to the loss of important information that the embedding features have extracted. To address this issue, we decided to replace the Multi-head Attention mechanism by using an MLP that includes a single linear layer. This change offers several key benefits:

- **Retention of feature information:** MLP allows us to retain and fully utilize the features extracted by the Code Embedding block without losing information due to the Attention process. With a single linear layer, the model can directly learn how to map from input vectors to desired outputs without creating complex links between different source codes.

- **Model simplification:** Eliminating Attention significantly reduces the complexity of the model, both in terms of structure and computation time. This not only makes the model easier to deploy but also enhances performance, especially when handling large datasets or real-time applications.

• **Reduction of noise from inconsistent labels:** MLP operates independently on each input vector, thus minimizing the risk of being affected by discrepancies in labels or distributions of source codes. This helps the model focus on extracting relevant features for each specific source code without attempting to create unnecessary connections between them. To evaluate the effectiveness of the new KA architecture, we will conduct a series of experiments comparing the MLP-based model with the Multi-head Attention-based model. The two main criteria we will consider are: Performance and Efficiency. First, regarding performance, we represent the KA block 7 similar to the traditional Transformer Encoder block (formulas below E are processed through Positional Encoding):

$$\widehat{\mathbf{E}} \in \mathbb{R}^{1 \times \widehat{d}} \quad (17)$$

$$\widehat{\mathbf{Q}} = \widehat{\mathbf{E}} * \widehat{\mathbf{W}}_{\mathbf{q}} + \mathbf{b}_{\widehat{\mathbf{q}}} \quad (18)$$

$$\widehat{\mathbf{K}} = \widehat{\mathbf{E}} * \widehat{\mathbf{W}}_{\mathbf{k}} + \mathbf{b}_{\widehat{\mathbf{k}}} \quad (19)$$

$$\widehat{\mathbf{V}} = \widehat{\mathbf{E}} * \widehat{\mathbf{W}}_{\mathbf{v}} + \mathbf{b}_{\widehat{\mathbf{v}}} \quad (20)$$

where:

$\widehat{\mathbf{W}}_{\mathbf{q}}, \widehat{\mathbf{W}}_{\mathbf{k}}, \widehat{\mathbf{W}}_{\mathbf{v}} \in \mathbb{R}^{\widehat{d} \times \widehat{n}}$ are the weight matrices of the linear layer.

• $\mathbf{b}_{\widehat{\mathbf{q}}}, \mathbf{b}_{\widehat{\mathbf{k}}}, \mathbf{b}_{\widehat{\mathbf{v}}}$ are the bias terms.

• $\widehat{n}$ is the dimension is defined by the following formula: $\widehat{n} = \left\lfloor \frac{\widehat{d}}{\text{num}} \right\rfloor$, where num is the number of Attention heads.

$$\widehat{\mathbf{S}} = \frac{\widehat{\mathbf{Q}} * \widehat{\mathbf{K}}^T}{\sqrt{\widehat{n}}} \in \mathbb{R}^{1 \times 1} \quad (21)$$

$$\widehat{\mathbf{B}} = [\mathbf{b}_{ij}] \in \mathbb{R}^{k \times n}, \mathbf{b}_{ij} = \frac{\exp(s_{ij})}{\sum_j \exp(s_{ij})} \rightarrow \widehat{\mathbf{B}} = [[1]] \quad (22)$$

$$\widehat{\mathbf{A}} = \text{Concat}(\widehat{\mathbf{B}} * \widehat{\mathbf{V}})_{\text{num}} * \mathbf{W} + \mathbf{b} \quad (23)$$

$$\widehat{\mathbf{A}} = \text{Concat}(\widehat{\mathbf{V}})_{\text{num}} * \mathbf{W} + \mathbf{b} \quad (24)$$

• $\mathbf{W} \in \mathbb{R}^{d \times \widehat{d}}, \mathbf{b} \in \mathbb{R}^{1 \times \widehat{d}}$ are the weight matrices and bias terms of the linear layer.

$$\widehat{\mathbf{A}} = \text{Concat}(\widehat{\mathbf{E}} * \widehat{\mathbf{W}} + \widehat{\mathbf{b}}) * \mathbf{W} + \mathbf{b} \quad (25)$$

• $\mathbf{W} \in \mathbb{R}^{d \times \widehat{d}}, \mathbf{b} \in \mathbb{R}^{1 \times \widehat{d}}$ are the weight matrices and bias terms of the linear layer.

$$\widehat{\mathbf{A}} = \widehat{\mathbf{E}} * \widehat{\mathbf{W}} + \widehat{\mathbf{b}} \quad (26)$$

• $\widehat{\mathbf{W}} \in \mathbb{R}^{d \times \widehat{d}}, \widehat{\mathbf{b}} \in \mathbb{R}^{1 \times \widehat{d}}$ are the weight matrices and bias terms of the linear layer.

Equation (25) essentially represents an MLP with two layers, where both layers use linear activation functions. This means that the network does not have non-linear activation functions such as ReLU or Sigmoid between layers, so the entire network performs only linear transformations on the input data. Since linear transformations are additive, applying multiple consecutive linear layers is equivalent to applying a single linear layer, leading to Eq. (26). This result shows that an MLP with multiple linear layers can be simplified to a network with just a single linear layer. This MLP offers several computational advantages over Multihead Attention. Firstly, in terms of computation speed, MLP does not require complex calculations as in Attention. Specifically, in the Attention mechanism, we need to compute three key matrices: the Query matrix (Q), the Key matrix (K), and the Value matrix (V). These matrices then go through complex calculations including matrix multiplication and applying the softmax function to compute weights for the values. Each of these steps requires high computational complexity, especially when the sizes of matrices Q, K, and V are large. In contrast, the computation of derivatives for the softmax function in Multi-head Attention is much more complex. Softmax is a non-linear function that converts input values into probabilities, and computing its derivative requires many more calculations, especially when applied to multiple inputs simultaneously. This not only increases computation time but can also cause numerical stability issues, particularly when input values vary greatly in magnitude. When applying the Attention mechanism to source code data, these complexities become even more pronounced. Source code data often have complex structures and varying lengths, making Attention computations less efficient. Additionally, source code from different systems or projects may carry various types of noise, making the Attention mechanism work harder to find meaningful links between code segments. This diversity in source code not only increases complexity but can also reduce model effectiveness, as Attention tends to focus on common patterns while ignoring rare but important details. On the other hand, MLP with a simpler structure maintains stability and higher efficiency when processing such source code data. MLP not only minimizes noise issues but also allows the model to focus on capturing the key features of each code segment without needing to compare them to other segments. This is particularly important when processing similar source code data, where MLP can perform significantly better than Multi-head Attention. Overall, an MLP with linear activation functions is not only simpler but also provides superior computational performance and efficiency compared to the Multi-head Attention mechanism. This makes it an ideal

choice for optimizing performance and stability during deep learning model training.

3.4 Building the KA layer

To design the KA layer, we focus on constructing its fundamental building blocks. Figure 2 below illustrates the components and operating principles of the KA Layer.

In Fig. 2, the structure of the KA layer includes key components such as MLP, FeedForward, and Add&Norm. We will not go into detail about the Positional Encoding section, as this part has been addressed in a manner similar to the approach in [42]. Starting with the MLP, this is a crucial component in the KA architecture. The MLP in KA is used to process and synthesize information from embedding vectors. The input to the MLP is a vector $\hat{E}$ with a dimensionality of $\hat{d}$, where $\hat{d}$ is the dimensionality for embedding the source code. The mathematical formula can be described below:

$$\mathbf{F} = \hat{\mathbf{E}} * \mathbf{W}, \mathbf{W} \in \mathbb{R}^{\hat{d} \times \hat{d}} \quad (27)$$

We have detailed the function of the MLP in the scientific basis section, explaining that the MLP allows the KA model to be less computationally expensive while achieving effectiveness comparable to the Multi-Head Attention block. We then pass the vector $\mathbf{F}$ through the Add&Norm block, with the formula illustrated below:

$$\mathbf{F} := \mathbf{F} + \mathbf{Y} \quad (28)$$

$$\mathbf{J} = \frac{\mathbf{F} - \mathbb{E}[\mathbf{F}]}{\sqrt{\text{Var}[\mathbf{F}] + \epsilon}} * \gamma + \beta \quad (29)$$

At this step, the model performs the addition $\mathbf{F} + \mathbf{Y}$ and normalizes the result to create the vector $\mathbf{J}$. Adding the vector $\mathbf{Y}$ is a crucial step in retaining information from the original representation, minimizing data loss during transformation. At the same time, normalizing this vector ensures that its values do not fluctuate excessively, helping to prevent the occurrence of gradient explosion or vanishing gradients during training. This approach makes the model more stable and enhances its ability to learn from complex data, even when the source code contains segments with heterogeneous structures. Next, we pass the vector $\mathbf{J}$ through the FeedForward block, with the formula illustrated below:

$$\mathbf{C} = \text{ReLU}(\mathbf{J} * \mathbf{W}_3), \mathbf{W}_3 \in \mathbb{R}^{\hat{d} \times P} \quad (30)$$

$$\mathbf{N} = \mathbf{C} * \mathbf{W}_4, \mathbf{W}_4 \in \mathbb{R}^{P \times \hat{d}} \quad (31)$$

The vector $\mathbf{J}$ is passed through the ReLU activation function and the FeedForward network. The ReLU activation function helps eliminate negative values, thus retaining more

essential information for feature recognition. This is particularly important in the context of source code vulnerability detection, as vulnerabilities are often subtle signals not easily noticeable in the raw data but can become apparent when nonlinear transformations are applied. ReLU also helps avoid the vanishing gradient problem, enabling faster and more efficient training, which in turn improves the model's accuracy in detecting vulnerabilities. We then pass the vector $\mathbf{N}$ through

the Add&Norm block, with the formula illustrated below:

$$\mathbf{N} := \mathbf{N} + \mathbf{J} \quad (32)$$

$$\mathbf{V} = \frac{\mathbf{N} - \mathbb{E}[\mathbf{N}]}{\sqrt{\text{Var}[\mathbf{N}] + \epsilon}} * \gamma + \beta \quad (33)$$

In the final stage, the vector $\mathbf{N}$ is normalized once more after being combined with the vector $\mathbf{J}$. This process ensures that every layer in the model maintains consistency in value ranges allowing subsequent layers to learn complex features without being impacted by excessive fluctuations from previous layers. Particularly in vulnerability detection, this makes the model more sensitive to anomalies in the source code. The final normalization step also helps the model retain generalization, avoiding overfitting on the training data and instead enabling flexible application to new source code.

3.5 Operation process of KA

The operational principle of the KA architecture involves a series of sequential steps designed to optimize the model's ability to process and learn features from source data. Each step in the process is carefully selected to ensure the extraction of essential features and to maintain model stability during training.

First, the data is passed through a Multi-Layer Perceptron (MLP), an architecture designed with dense and deep layers to extract core information from each segment of source code. Unlike models based on Multi-Head Attention (MHA), the use of MLP here is due to the input data being single vectors instead of sequential matrices. Therefore, MLP use simplifies computation, conserves resources, and reduces noise that MHA may occasionally encounter when processing non-sequential data. MLP helps the model focus on key features, efficiently filtering out unnecessary elements.

Once the data is aggregated through the MLP, the information is fed into the FeedForward block, where deep learning continues. This block includes nonlinear activation functions, allowing the model to explore and learn more complex features from the data. The choice to include a FeedForward block rather than stopping at MLP is due to the need for higher-order feature learning. When data passes through nonlinear activation functions like ReLU or GELU, complex nonlinear patterns within the data are captured in more detail,

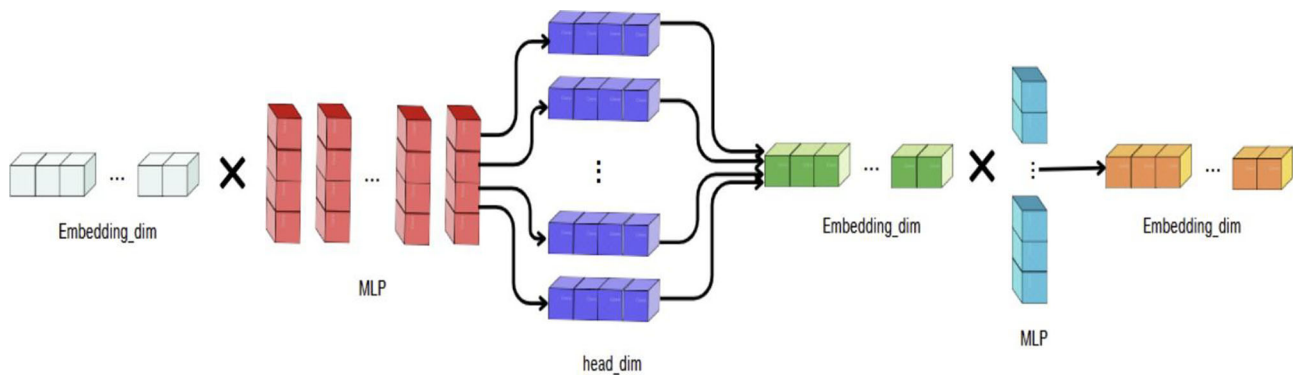


Fig. 2 KA layer architecture

enabling the model to develop a deeper understanding of data structures that a single MLP layer alone may not be able to achieve.

Finally, the data passes through the Add&Norm block (Residual Connection and Layer Normalization). This method not only helps maintain model stability but also enhances continuous learning by adding residual connections. These connections ensure the model does not “forget” important features from previous steps, while also helping prevent gradient degradation during training. Layer Normalization further ensures stability in activation values, allowing the model to learn effectively even when input data has a complex distribution. The KA architecture is built to optimize performance when processing source data features, particularly by reducing computational costs and increasing model efficiency while preserving essential information. Although the traditional transformer encoder, which uses MHA, is known for its ability to model positional relationships within sequences, the KA architecture opts for a more streamlined approach through MLP to focus on key features and limit processing of unnecessary elements. While transformer encoders often struggle to process long data sequences due to memory requirements and high computational costs, the KA architecture mitigates this issue by focusing on processing data through single vectors. This approach not only conserves resources but also helps the model maintain stability and accuracy even when applied to large code datasets. Rather than relying on all relationships among words in a lengthy sequence, KA aims to capture the core characteristics of code vectors, a feature more suitable for source code data than natural language data, where relationships between words in the sequence play a more critical role. Eliminating MHA and replacing it with MLP not only simplifies the structure but also allows KA to achieve higher efficiency when applied to code segments with certain structures, while still retaining the ability to learn key features. Additionally, the KA architecture integrates a FeedForward block with non-linear activation functions like ReLU or GELU, enabling the

modeling of higher-order features from data. As a result, the model can still capture complex characteristics within source code without requiring large memory or increased computation time as sequence lengths grow. With these optimizations, the KA architecture is well-equipped to handle large data sequences efficiently, leveraging MLP to retain critical features in source code data without relying on complex long-context sequence processing methods like MHA.

3.6 Classification block for FG-CVD

After the source code feature vectors have been transformed and processed through the Embedding Coder and KA blocks, this study selects the use of MLP for classification to achieve the highest efficiency. MLP is chosen for its ability to handle complex features and its flexibility when working with encoded inputs, which aligns well with the model’s architecture. During experimentation, we also tested the Classification block with other approaches for comparison, including RandomForest, CNN, and Mean-Pooling. RandomForest is highly regarded for its stability and decision-making based on multiple classification trees, while CNN is advantageous in extracting spatial features, and MeanPooling offers an effective data dimensionality reduction method by taking the average value of the vectors. However, MLP demonstrated superior performance in learning and classifying source code features, particularly when these features have been processed through the Embedding and KA blocks, enabling the model to achieve higher accuracy and stability in comparative experiments.

4 Experiments and evaluation

4.1 Experimental data

To evaluate the proposed model, we utilized four primary datasets: FFMQ+QEMU [44], Reveal [42], Bigvul [41], and

Table 3 Table comparing the total number and percentage of vulnerabilities

Dataset	Total	vulnerability rate
FFMQ + QEMU[44]	27,318	45.6
Reveal[42]	18,169	9.16
BigVul[41]	216,635	4.95
RealVul[43]	270,919	2.04

Table 4 The configuration of the proposed model's hyper-parameters

Model	Hyper-parameter	Value
Encoder	max_length	512
	dim_encode	768
	dim_output	512
	num_layer_mlp	4
KA	num layer KA	12
	dim_feedforward	768
	dim_output	512
Training	num_train_epochs	9
	gradient_accumulation_steps	12
	learning_rate	3.10^{-5}

Realvul [43]. Table 3 below describes the components of the experimental datasets used in this study.

4.2 Evaluation criteria and environment

4.2.1 Evaluation criteria

We applied four key metrics commonly used in this field of research:

$$accuracy = \frac{TP + TN}{TP + TN + FP + FN} \times 100\%$$

$$precision = \frac{TP}{TP + FP} \times 100\%$$

$$Recall = \frac{TP}{TP + FN} \times 100\%$$

$$F1 = \frac{2 \times precision \times Recall}{precision + Recall}$$

where:

TP—True Positive: The number of vulnerable samples classified correctly.

FN—False Negative: The number of vulnerable samples classified as non-vulnerable.

TN—True Negative: The number of non-vulnerable samples classified correctly.

FP—False Positive: The number of non-vulnerable samples classified as vulnerable.

5 Experimental environment

The experimental evaluation was conducted on a high performance computing platform featuring an NVIDIA Tesla T4 GPU with 2,560 CUDA cores and 16 GB of GDDR6 memory, selected for its optimal balance between computational capability and memory capacity. To ensure comprehensive model assessment, we systematically evaluated various hyperparameter configurations. Table 4 presents the detailed hyperparameter settings employed in our experimental framework.

5.1 Experimental scenarios

To demonstrate the effectiveness of the proposed model, the paper will conduct experiments according to the following research questions:

Research question 1 (RQ1): How well does the proposed model perform on experimental datasets with different structures and characteristics?

Answer research question (ARQ1): To address this question, the paper will conduct experiments on the datasets listed in Table 3. Significant differences among these experimental datasets include: first, their structure and characteristics; second, the quantity and ratio of labels.

RQ2: Does this model perform effectively when deployed in real-world scenarios?

ARQ2: To answer this question, we will perform cross-validation of the models by training with this model but using different models for testing

5.2 Results for RQ1

The experimental results presented in Table 5 below show the evaluation and testing outcomes of the proposed model in comparison with several other studies using the datasets listed in Table 3

The experimental results presented in Table 5 demonstrate the significant performance advantages of the FG-CVD model across multiple datasets. On the balanced FFMQ + QEMU dataset, FG-CVD exhibits superior performance across all evaluation metrics compared to existing approaches. While AMPLE achieves a marginally higher recall rate (by 0.8%), FG-CVD's superior performance across other metrics established it as the optimal solution for vulnerability detection tasks. For imbalanced datasets, FG-CVD demonstrates remarkable effectiveness. Notably, on the Reveal dataset, characterized by a 1:10 imbalance ratio, conventional methods without data balancing typically underperform by 2–20%. However, FG-CVD achieves comparable or superior results to models employing specialized data balancing techniques, such as GRD [20] and Reveal [42], across multiple performance metrics, despite not implementing explicit data balancing mechanisms. Furthermore,

Table 5 Experimental results evaluating the FC-CVD model on four datasets: FFMQ, REVEAL, BigVul, RealVul

Dataset	Model	Acc	Pre	Rec	F1
FFMQ + QEMU	GRD[20]	62.78	57.15	72.98	64.10
	AMPLE[48]	62.16	55.64	83.99	66.94
	Reveal[42]	62.51	56.85	72.61	64.42
	MAGNET[45]	63.28	56.27	80.15	66.12
	LIVABLE[46]	64.84	57.87	80.67	67.39
	SySeVR [47]	52.52	48.34	65.96	56.03
	Our	65.4	57.51	83.14	67.99
REVEAL	GRD (re)[20]	87.72	39.42	67.07	49.65
	ReVeal (re)[42]	86.94	34.03	64.24	44.49
	GRD (im)[20]	87.24	38.36	64.40	48.08
	ReVeal (im)[20]	90.48	46.23	15.09	22.44
	AMPLE (im)[48]	92.71	51.05	46.15	48.48
	MAGNET(im)[45]	91.60	42.86	61.68	50.57
	LIVABLE(im)[46]	93.53	52.27	50.55	51.50
	SySeVR (im)[47]	84.22	24.46	40.11	30.25
	Our (im)	90.03	46.49	61.78	53.05
BigVul	SySeVR[47]	90.10	30.91	14.08	19.34
	AMPLE[48]	93.14	29.98	34.58	32.11
	Reveal (re)[42]	87.14	17.22	30.61	22.87
	MAGNET[45]	91.38	22.71	38.92	28.68
	LIVABLE[46]	95.05	40.04	37.27	38.60
	Our	96.19	64.49	51.63	57.34
RealVul	DeepWukong[29]	99	0	0	0
	LineVul[32]	99	0	0	0
	ReVeal (re)[42]	95	2	10	3
	IVDetect[30]	81	2	1	0
	Our	84.89	3.4	60.37	6.4

the FG-CVD model demonstrates exceptional performance on large-scale, highly imbalanced datasets: BigVul [41] and RealVul [43], which exhibit imbalance ratios of 1:25 and 1:50 respectively. The model not only surpasses conventional approaches without data balancing but also outperforms methods employing specialized balancing techniques. Despite the inherent challenges in classifying large-scale imbalanced datasets, FG-CVD effectively addresses these limitations through its robust processing architecture. The following sections present a comprehensive analysis of the experimental results across individual datasets.

source code vulnerability classification. The model achieves significant performance metrics: Accuracy (65.4%), Precision (57.51%), Recall (83.14%), and F1-score (67.99%). These results surpass traditional approaches including GRD, Reveal, MAGNET, and SySeVR across all metrics, with improvements ranging from 1–19%. While experiencing marginal decreases in Precision compared to AMPLE [48] (0.8%) and LIVABLE [46] (0.3%), FG-CVD demonstrates substantial improvements of 1–3% across other performance metrics. These results validate the model's effectiveness in maintaining both high detection capability and optimal balance between Precision and Recall—critical factors in vulnerability classification tasks.

6 Comments and evaluation

6.1 Firstly, for the FFMQ dataset

The experimental results presented in Table 5 demonstrate the superior performance of our proposed FG-CVD model in

6.2 Secondly, for the REVEAL dataset

The experimental results show that our model achieves outstanding performance without the need for data generation or

balancing methods. Specifically, our model achieves Accuracy, Precision, Recall, and F1-score metrics of 90.03, 46.49, 61.78, and 53.05% respectively. This demonstrates the strong and stable classification capability of the model even in the case of imbalanced data.

A detailed comparison with other models is as follows: GRD (rebalancing data): Although GRD [20] uses Dropout for data generation, this model only achieves an F1-score of 49.65% and Recall of 67.07%, which is not significantly different from the FG-CVD model. This proves that our model can achieve comparable performance without applying data balancing techniques. ReVeal (rebalancing data): ReVeal also uses SMOTE for data generation, achieving an F1-score of 44.49%. Our model outperforms with an F1-score nearly 6% higher. SySeVR [47], AMPLE [48], MAGNET [45], LIVABLE [46]: For these methods that do not use data balancing, our model has outperformed in all metrics and provides a good balance across metrics. Overall, the results confirm that our model not only maintains high accuracy (Accuracy) but also has a more effective ability to detect and classify positive labels (Recall) compared to many other models, even without using data balancing methods. This emphasizes that our model can perform excellently in real-world scenarios without requiring data adjustment, saving time and resources during deployment.

6.3 Thirdly, for the Bigvul dataset

The BigVul dataset [41], characterized by its large scale and significant label imbalance ratio, provides a challenging evaluation environment. Our FG-CVD model demonstrates exceptional capability in handling these imbalances, outperforming existing methods by margins of 9–45% across all evaluation metrics. Notably, while Reveal's SMOTE-based approach shows effectiveness on smaller datasets, it exhibits performance degradation on the more challenging BigVul dataset. However, it is important to acknowledge that despite BigVul's extensive collection of real-world C/C++ code, Croft et al. [34] identified accuracy limitations in the dataset, reporting only 54.3% accuracy in their validation study. This inherent dataset limitation presents significant challenges in achieving optimal experimental outcomes.

6.4 Fourthly, for the Realvul dataset

The RealVul dataset presents the most challenging evaluation scenario, with an unprecedented imbalance ratio of 1:50. Previous research [43] has documented the significant performance limitations of existing models on this dataset. Our FG-CVD model, however, achieves superior performance across precision, recall, and F1 metrics compared to current approaches. While the Reveal model [42], utilizing SMOTE based data balancing, demonstrates moderate improvements

over baseline methods, its performance remains suboptimal. Notably, FG-CVD achieves these superior results without implementing explicit data balancing techniques, highlighting the robustness of our architectural design in handling extreme data imbalances.

Conclusion: The results from Table 5 clearly indicate two issues that current models face, which other studies, despite proposing various techniques, have not yet addressed: large datasets and imbalanced datasets. Clearly, data generation and balancing methods only provide effective results for some datasets, especially smaller ones. Balancing might be reconsidered as a good way to enhance model performance, but it still does not suit large datasets with high imbalance ratios. The issue is that data balancing techniques, when generating data, also produce erroneous data, leading to noise in the classification model. 2) Reasons and Solutions The consistent superior performance of FG-CVD across diverse datasets warrants a detailed analysis of its underlying mechanisms. We hypothesize that the synergistic integration of the embedding code model and the novel Knowledge Augmentation (KA) architecture enhances the model's feature extraction and synthesis capabilities. To validate this hypothesis, we conduct a comprehensive analysis of data representations within the FG-CVD framework, specifically examining the data distribution transformations before and after KA processing. Our analysis focuses on two distinct scenarios: balanced data distribution using the FFMQ dataset, and imbalanced data distribution using the BigVul dataset, enabling a thorough evaluation of the model's processing capabilities under different data conditions.

7 Reasons and solutions

The consistent superior performance of FG-CVD across diverse datasets warrants a detailed analysis of its underlying mechanisms. We hypothesize that the synergistic integration of the embedding code model and the novel Knowledge Augmentation (KA) architecture enhances the model's feature extraction and synthesis capabilities. To validate this hypothesis, we conduct a comprehensive analysis of data representations within the FG-CVD framework, specifically examining the data distribution transformations before and after KA processing. Our analysis focuses on two distinct scenarios: balanced data distribution using the FFMQ dataset, and imbalanced data distribution using the BigVul dataset, enabling a thorough evaluation of the model's processing capabilities under different data conditions. Figure 3 illustrates the detailed data distribution of the two experimental datasets.

Analysis of Fig. 3 reveals significant improvements in class separation through our enhanced transformer architecture. In the initial representation, the binary classes (denoted

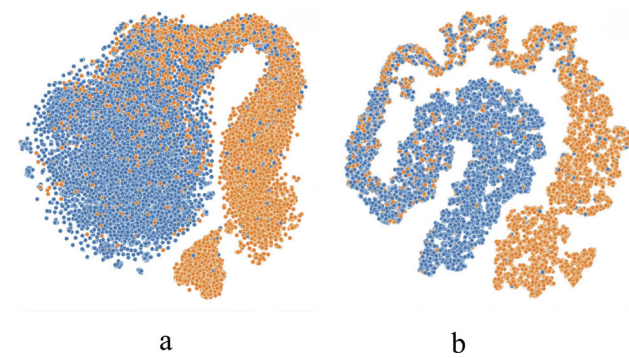


Fig. 3 Data distribution of FFMQ before and after processing with KA, where **a** is before applying KA and **b** is after applying KA. The blue points are normal points, and the orange points are vulnerability points

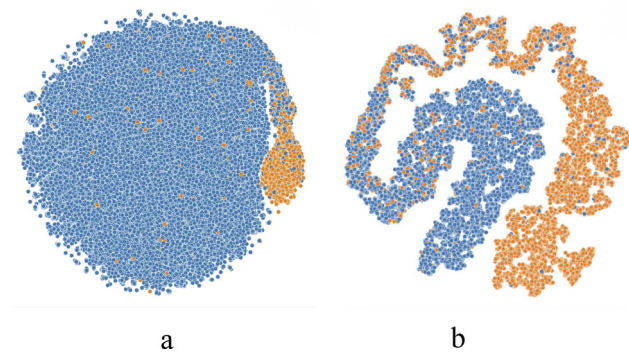


Fig. 4 Distribution of BigVul data (imbalanced dataset) before and after processing with KA, where **a** is before using KA and **b** is after using KA. The blue points are normal points, and the orange points are vulnerability points

by blue and orange) exhibit substantial overlap, particularly in the central distribution region, indicating insufficient discriminative power in the original feature space. This overlap suggests that the initial feature representations lack the necessary discriminative characteristics for effective class separation. Post-processing by our enhanced transformer demonstrates marked improvement in class differentiation, with distinct cluster formation and minimal inter-class overlap. This transformation evidences the enhanced model's capability to generate more discriminative feature representations, facilitating improved classification performance through better-defined decision boundaries between classes (Fig. 4).

Similarly, before processing through the improved Transformer, it can be observed that the data of the two classes blue and orange are not only not clearly separated but also exhibit an imbalance between the two classes. The minority class may be obscured by the larger class, making it difficult to identify and distinguish the points of the minority class. This data imbalance makes it challenging for the model to learn the important features of the minority class. As a result, the model may become biased towards the larger class, leading to incorrect predictions for samples from

the minority class. After processing the data through the improved Transformer, the data of the two classes is more clearly separated, although the imbalance still exists. The improved Transformer has enhanced the representation of the minority class, making the data points of this class more recognizable and not completely obscured by the larger class. The improved Transformer has helped the model learn better from the imbalanced data, improving the discrimination between classes, especially the minority class. This enhancement increases classification performance, reduces bias, and improves accuracy for the minority class.

7.1 Evaluation results for RQ2

To rigorously evaluate our model's generalization capabilities, we implement a comprehensive cross-dataset validation framework. This approach is particularly crucial given the findings of [43], which highlight significant reliability concerns in vulnerability labeling across existing datasets—a challenge stemming from the resource-intensive nature of expert-based manual annotation. Our cross-validation framework encompasses three distinct scenarios: (1) training on FFMQ with RealVul validation, (2) training on FFMQ with ReVeal validation, and (3) hybrid training using merged FFMQ and ReVeal datasets. This experimental design addresses two critical research questions: the model's robustness across heterogeneous dataset architectures, and its effectiveness in handling merged datasets with diverse structural characteristics. Table 6 below lists the experimental results of this scenario.

The experimental results in Table 6 provide the following information:

The first, in the experiment using the FFMQ dataset for training and the RealVul dataset for testing, the results yielded an Accuracy of 95.41%, Precision of 60.45%, Recall of 41.19%, and an F1-Score of 48.99%. Due to the differences between the datasets, the RealVul dataset used for testing contains only 5% of the samples related to the FFMQ + QEMU dataset and has a label imbalance of up to 50 times. Despite this, our model still achieved good results in vulnerability detection. These results are quite acceptable given the nature of the code in the test set differing by 95% from the training dataset. This addresses the challenge of having sufficiently large and reliable datasets for accurate labeling. This is a promising result for using the model to detect zero-day vulnerabilities.

The second validation scenario, utilizing FFMQ for training and Reveal for testing, further validates our initial findings. Despite minimal correlation between datasets, the model maintains reasonable performance metrics. The observed high Recall coupled with lower Precision indicates increased false positive rates, a common trade-off in security-critical applications. Comparative analysis with

Table 6 Cross-Validation Experimental Results

Cross-Validation Method	Acc	Pre	Rec	F1
Using FFMQ for training and RealVul for testing	95.41	60.45	41.19	48.99
Using FFMQ for training and Reveal for testing	32.51	32.49	95.11	48.44
Mixing Reveal and FFMQ datasets	95.34	59.74	40.09	47.98

direct ReVeal training–testing scenarios shows performance comparable to SySeVR [47], demonstrating the model’s stability across different evaluation contexts.

The final experiment involved training on a merged FFMQ ReVeal dataset, deliberately introducing structural heterogeneity to evaluate model robustness. Despite this challenging scenario, the model achieved remarkable performance: 95.34% Accuracy, 59.74% Precision, 40.09% Recall, and 47.98% F1 Score. These metrics demonstrate optimal balance between precision and recall, crucial for practical vulnerability detection applications, validating the effectiveness of our proposed architecture.

8 Discussion

The experimental results in Sect. 4 have demonstrated the effectiveness of the proposed model. In this section, we will provide some evidence to further clarify the contribution of each component in the model as well as some limitations that need to be addressed.

8.1 Evaluation of the encoder’s role

The experimental results above show that the FG-CVD model has performed well in the task of encoding data. However, the question remains whether this Encoder model is the best or most optimal. To address this question, we will replace the Encoder in FG-CVD with several other Encoder models such as CodeBERT, RoBERTa, and CodeSage-small. The Table 7 below lists the results of this comparison.

The experimental results show that the proposed model in the paper outperforms other models. Particularly, the Recall metric is notably better. We observed that Recall on the FFMQ dataset is generally higher compared to other datasets. However, recent studies indicate that achieving high Recall often results in a significant decrease in other metrics. Nonetheless, the results of our model demonstrate that all four metrics are improved and stable.

8.2 Evaluation of the role of KA model

To evaluate the role and effectiveness of KA, we will replace this model with several traditional networks such as Attention, Mean Pooling, etc. Table 8 below lists the comparison and evaluation results of the KA model with other approaches.

From these results (see Table 8), it is evident that our proposed transformer improvement method outperforms all other metrics. This clearly demonstrates the suitability of KA for the problem.

8.3 Threats to validity

Despite the demonstrated superior performance of our proposed model across multiple scenarios, several limitations warrant discussion:

8.3.1 Dataset dependency

Experimental results reveal significant performance variations across different datasets. While some datasets show modest improvements of 2–10% compared to existing studies, others demonstrate improvements of several orders of magnitude. This variability primarily stems from the model’s inherent dependency on dataset labeling quality. Recent research has highlighted critical labeling accuracy concerns: Croft et al. [34] identified numerous instances of benign functions incorrectly labeled as vulnerable in the BigVul dataset, while Zhou et al. [43] documented widespread label inconsistencies across multiple vulnerability datasets. These labeling inaccuracies significantly impact both training and evaluation processes.

8.3.2 Model limitations and future directions

Our experimental analysis reveals two significant limitations. First, the model exhibits substantial performance variability across different datasets, with improvements ranging from modest (2–10%) to several orders of magnitude compared to existing approaches. This variability primarily stems from dataset labeling quality dependencies, as evidenced by recent research: Croft et al. [34] identified numerous misclassified

Table 7 Results of Replacing the Encoder with Various Models

Model	Acc	Pre	Rec	F1
CodeBERT [10]+ KA	65.11	59.56	73.06	65.62
RoBERTa [12]+ KA	63.01	57.87	69.39	63.11
CodeSage [51]+ KA	64.83	57.04	74.59	64.46
Our	65.4	57.51	83.14	67.99

Table 8 Performance of KA with Different Networks

Model	Acc	Pre	Rec	F1
Attention	63.3	57.4	70.4	63.2
Mean Pooling	62.5	51.4	65.6	57.63
KA	65.4	57.51	83.14	67.99

benign functions in the BigVul dataset, while Zhou et al. [43] documented pervasive label inconsistencies across vulnerability datasets. Second, while cross-validation results (Table 7) demonstrate superior overall performance metrics, the precision metrics fall short of production-ready standards by an estimated 10–15%, particularly in imbalanced datasets. Traditional remediation approaches include data augmentation techniques (SMOTE [42], CGAN [11]), regularization methods (Dropout [20]), and advanced learning paradigms (contrastive and metric learning). Integration of these optimization techniques represents a promising direction for future research iterations.

9 Conclusion

Early detection and warning of source code vulnerabilities is and will be a challenging issue for software verification solutions, as attackers continuously seek to modify and create new zero-day vulnerabilities. In this paper, with the goal of providing a new solution and model that performs effectively across various datasets, we have successfully proposed the FG CVD model. Based on the experimental results in the proposed scenarios, the paper has provided readers with a comprehensive and insightful view of how datasets impact detection models. Another significant contribution of this paper is the evidence and references on how to build an effective source code vulnerability detection model based on LLMs. Finally, the paper indicates that LLMs are not a magic solution that can address all issues in source code vulnerability detection. The most crucial factor remains the data characteristics. Therefore, to enhance the practical effectiveness of detection models, it is necessary to flexibly combine LLMs with various analysis techniques and models. This approach not only leverages the advantages of each model but also minimizes their drawbacks. The questions of how and in what way to combine will always remain open and

have no optimal solution yet. In the future, to further improve the effectiveness of this paper, we believe that attention should be focused on addressing a few key stages: first, in the code embedding stage, instead of feeding the entire source code into the FG-CVD model, additional models such as summarization or code compression can be used to find representative versions of code segments using Seq2Seq or transformer models like GPT, CodeX, and Attention. Next, in the stage of highlighting important information and classification, data balancing models can be incorporated, combined with methods such as contrastive learning.

Acknowledgements This work has been sponsored by the Posts and Telecommunications Institute of Technology and Ministry of Information and Communications, Vietnam.

Authors' contribution CDX raised the idea, initialized the project, and designed the experiments; VDQ and DBQ carried out the experiments under the supervision of CDX. Both authors analyzed the data and results. CDX wrote the paper.

Funding No funding was received for this work.

Availability of data and materials The datasets generated and/or analyzed during the current study are available from the corresponding author on reasonable request. No datasets were generated or analysed during the current study.

Declarations

Conflict of interest We wish to confirm that there are no known conflicts of interest associated with this publication and there has been no significant financial support for this work that could have influenced its outcome.

Ethical approval Not applicable.

References

1. Lin, G., Wen, S., Han, Q.-L., Zhang, J., Xiang, Y.: Software vulnerability detection using deep neural networks: a survey. In: Proceedings of the IEEE, 108(10), 1825–1848 (2020)

2. Stubborn Weaknesses in the CWE Top 25. Accessed: 2024. [Online].
3. Open Source Security and Risk Analysis Report (OSSRA) Synopsis. <https://www.blackduck.com/resources/analyst-reports/open-source-security-risk-analysis.html#introMenu>. Accessed: 2024. [Online].
4. Tian, Z., Tian, B., Lv, J., Chen, Y., Chen, L.: Enhancing vulnerability detection via AST decomposition and neural sub-tree encoding. *Exp. Syst. Appl.* **238**(part B). Art. no. 121865, ISSN 0957-4174 (2024)
5. Lin, G. et al.: Software vulnerability discovery via learning multi-domain knowledge bases. *IEEE Trans. Depend. Secure Comput.* **18**(5), 2469–2485 (2021)
6. Xu, R., Tang, Z., Ye, G., Wang, H., Ke, X., Fang, D., Wang, Z.: Detecting code vulnerabilities by learning from large-scale open source repositories. *J. Inf. Security Appl.* **69**, Art. no. 103293, ISSN 2214-2126 (2022)
7. Kande, R., et al.: (Security) assertions by large language models. *IEEE Trans. Inf. Forensics Secur.* **19**, 4374–4389 (2024)
8. Chakraborty, S., Ahmed, T., Ding, Y., Devanbu, P.T., Ray, B.: NatGen: generative pre-training by “naturalizing” source code. In: *Proceedings of the 30th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering*, pp. 18–30 (2022)
9. Raffel, C., Shazeer, N., Roberts, A., Lee, K., Narang, S., Matena, M., Zhou, Y., Li, W., Liu, P.J.: Exploring the limits of transfer learning with a unified text-to-text transformer. *J. Mach. Learn. Res.* **21**(1), 1–67 (2020)
10. Wang, Y., Wang, W., Joty, S.R., Hoi, S.C.: CodeT5: identifier-aware unified pre-trained encoder-decoder models for code understanding and generation. *arXiv preprint arXiv:2109.00859* (2021)
11. Feng, Z., Guo, D., Tang, D., Duan, N., Feng, X., Gong, M., Shou, L., Liu, T., Jiang, D., Zhou, M.: CodeBERT: A Pre-Trained Model for Programming and Natural Languages (2020)
12. Zheng, N., Ding, J., Chai, T.: DMGAN: adversarial learning-based decision making for human-level plant-wide operation of process industries under uncertainties. *IEEE Trans. Neural Netw. Learn. Syst.* **32**(3), 985–998 (2021)
13. Liu, Y., Ott, M., Goyal, N., Du, J., Joshi, M., Chen, D., Levy, O., Lewis, M., Zettlemoyer, L., Stoyanov, V.: RoBERTa: A robustly optimized BERT pretraining approach. *arXiv:1907.11692* (2019)
14. Koschke, R., Falke, R., Frenzel, P.: Clone detection using abstract syntax suffix trees. In: *Proceedings of the 13th Working Conference on Reverse Engineering*, Benevento, Italy, pp. 253–262 (2006)
15. Yamaguchi, F., Golde, N., Arp, D., Rieck, K.: Modeling and discovering vulnerabilities with code property graphs. In: *Proceedings of the 2014 IEEE Symposium on Security and Privacy*, Berkeley, CA, USA, pp. 590–604 (2014)
16. Lu, G., Ju, X., Chen, X., Pei, W., Cai, Z.: GRACE: Empowering LLM-based software vulnerability detection with graph structure and in-context learning. *J. Syst. Softw.* **212**, 112031. ISSN 0164-1212 (2024)
17. Liu, Z., Tang, Y., Luo, X., Zhou, Y., Zhang, L.F.: No need to lift a finger anymore? Assessing the quality of code generation by ChatGPT. *IEEE Trans. Software Eng.* **50**(6), 1548–1584 (2024)
18. Chen, Y., Ding, Z., Alowain, L., Chen, X., Wagner, D.: DiverseVul: a new vulnerable source code dataset for deep learning based vulnerability detection. In: *Proceedings of the 26th International Symposium on Research in Attacks, Intrusions and Defenses (RAID '23)*, Association for Computing Machinery, New York, NY, USA, pp. 654–668 (2023)
19. Kumar, P.: Adversarial attacks and defenses for large language models (LLMs): methods, frameworks and challenges. *Int. J. Multimed. Info. Retr.* **13**, 26 (2024)
20. Saiyedand, M.F., Al-Anbagi, I.: Deep ensemble learning with pruning for DDoS attack detection in IoT networks. *IEEE Trans. Mach. Learn. Commun. Netw.* **2**, 596–616 (2024)
21. Bugshan, N., Khalil, I., Kalapaaking, A.P., Atiquzzaman, M.: Intrusion detection-based ensemble learning and microservices for zero touch networks. *IEEE Commun. Mag.* **61**(6), 86–92 (2023)
22. Do Xuan, C., Cong, B.V.: An advanced computing approach for software vulnerability detection. *Multimed. Tools Appl.* (2024)
23. Kong, L., Luo, S., Pan, L., Wu, Z., Li, X.: A multi-type vulnerability detection framework with parallel perspective fusion and hierarchical feature enhancement. *Comput. Secur.* **140**, 103787 (2024)
24. Thakkar, A., Lohiya, R.: Attack classification of imbalanced intrusion data for IoT network using ensemble-learning-based deep neural network. *IEEE Internet of Things J.* **10**(13), 11888–11895 (2023)
25. Tang, W., Tang, M., Ban, M., Zhao, Z., Feng, M.: CSGVD: A deep learning approach combining sequence and graph embedding for source code vulnerability detection. *J. Syst. Softw.* **199**, 111623 (2023)
26. Cheng, X., Nie, X., Li, N., Wang, H., Zheng, Z., Sui, Y.: How about bug-triggering paths?—understanding and characterizing learning-based vulnerability detectors. *IEEE Trans. Dependable Sec. Comput.*, **21**(2), 542–558 (2024)
27. Wang, H., et al.: Combining graph-based learning with automated data collection for code vulnerability detection. *IEEE Trans. Inf. Forensics Security* **16**, 1943–1958 (2021)
28. Li, W., Dou, S., Wu, Y., Li, C., Liu, Y.: COCL: an intelligent framework for enhancing deep learning-based vulnerability detection. *IEEE Trans. Ind. Informat.* **20**(3), 4953–4961 (2024)
29. J. Wang, H.X., Zhong, S., Xiao, Y.: DeepVulSeeker: a novel vulnerability identification framework via code graph structure and pre-training mechanism. *Future Gener. Comput. Syst.*, **148**, pp. 15–26 (2023)
30. Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A.N., Kaiser, Ł., Polosukhin, I.: Attention is all you need. In: *Proceedings of the 31st International Conference on Neural Information Processing Systems (NIPS'17)*, Curran Associates Inc., Red Hook, NY, USA, 2017, pp. 6000–6010.
31. Cheng, X., Wang, H., Hua, J., Xu, G., Sui, Y.: DeepWukong: statically detecting software vulnerabilities using deep graph neural network. *ACM Trans. Softw. Eng. Methodol.*, **30**(3). Art. 38 (2021)
32. Fu, M., Tantithamthavorn, C.: LineVul: a transformer-based line-level vulnerability prediction. In: *2022 IEEE/ACM 19th International Conference on Mining Software Repositories (MSR)*, Pittsburgh, PA, USA, pp. 608–620 (2022)
33. Li, Y., Wang, S., Nguyen, T.N.: Vulnerability detection with fine-grained interpretations. In: *Proceedings of the 29th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering (ESEC/FSE 2021)*, Association for Computing Machinery, New York, NY, USA, pp. 292–303 (2021)
34. Guo, D., Lu, S., Duan, N., Wang, Y., Zhou, M., Yin, J.: UniXcoder: unified cross-modal pre-training for code representation. In: *Annual Meeting of the Association for Computational Linguistics (ACL 2022)*
35. Yang, Z., Zhao, Z., Wang, C., Shi, J., Kim, D., Han, D., Lo, D.: Unveiling memorization in code models. In: *Proceedings of the IEEE/ACM 46th International Conference on Software Engineering (ICSE 2024)*, pp. 1–13 (2024)
36. Wang, Y., Le, H., Gotmare, A.D., Bui, N.D.Q., Li, J., Hoi, S.C.H.: CodeT5+: open code large language models for code understanding and generation. In: *Proceedings of the Conference on Empirical Methods in Natural Language Processing (EMNLP 2023)*, (2023)
37. Croft, R., Babar, M.A., Kholoosi, M.M.: Data quality for software vulnerability datasets. In: *Proceedings of the IEEE/ACM*

- 45th International Conference on Software Engineering (ICSE), pp. 121–133 (2023)
38. Wu, T., Chen, L., Du, G., Meng, D., Shi, G.: UltraVCS: ultra-fine-grained variable-based code slicing for automated vulnerability detection. *IEEE Trans. Inf. Forensics Secur.* **19**, 3986–4000 (2024)
 39. Cao, Y., Jiang, F., Xiao, J., Chen, S., Shao, X., Wu, C.: SCcheck: a novel graph-driven and attention-enabled smart contract vulnerability detection framework for web 3.0 ecosystem. *IEEE Trans. Netw. Sci. Eng.* **11**(5), 4007–4019
 40. Liu, S., Lin, G., Han, Q.-L., Wen, S., Zhang, J., Xiang, Y.: Deep-Balance: deep-learning and fuzzy oversampling for vulnerability detection. *IEEE Trans. Fuzzy Syst.* **28**(7), 1329–1343 (2020)
 41. Ahmad, B., Thakur, S., Tan, B., Karri, R., Pearce, H.: On hardware security bug code fixes by prompting large language models. *IEEE Trans. Inf. Forensics Secur.* **19**, 4043–4057 (2024)
 42. Yi, X., Wu, J., Li, G., Bashir, A.K., Li, J., AlZubi, A.A.: Recurrent semantic learning-driven fast binary vulnerability detection in healthcare cyber physical systems. *IEEE Trans. Netw. Sci. Eng.* **10**(5), 2537–2550
 43. Zhang, Y. et al.: An efficient smart contract vulnerability detector based on semantic contract graphs using approximate graph matching. *IEEE Internet Things J.* **10**(24), 21431–21442 (2023)
 44. Fan, J., Li, Y., Wang, S., Nguyen, T.N.: A C/C++ code vulnerability dataset with code changes and CVE summaries. In: *Proceedings of the 17th International Conference on Mining Software Repositories (MSR '20)*, New York, NY, USA, pp. 508–512 (2020)
 45. Chakraborty, S., Krishna, R., Ding, Y., Ray, B.: Deep learning based vulnerability detection: Are we there yet? *IEEE Trans. Software Eng.* **48**(9), 3280–3296 (2022)
 46. Chakraborty, P., Arumugam, K.K., Alfadel, M., Nagappan, M., McIntosh, S.: Revisiting the performance of deep learning-based vulnerability detection on realistic datasets. *IEEE Trans. Software Eng.* **50**(8), 2163–2177 (2024)
 47. Zhou, Y., Liu, S., Siow, J., Du, X., Liu, Y.: Devign: effective vulnerability identification by learning comprehensive program semantics via graph neural networks. In: *Proceedings of the 33rd International Conference on Neural Information Processing Systems*, Curran Associates Inc., Red Hook, NY, USA, 2019, Article 915, pp. 10197–10207 (2019)
 48. Wen, X.-C., et al.: Meta-path based attentional graph learning model for vulnerability detection. *IEEE Trans. Software Eng.* **50**(3), 360–375 (2024)
 49. Wen, X.-C., Gao, C., Luo, F., Wang, H., Li, G., Liao, Q.: LIVABLE: exploring long-tailed classification of software vulnerability types. *IEEE Trans. Software Eng.* **50**(6), 1325–1339 (2024)
 50. Li, Z., Zou, D., Xu, S., Jin, H., Zhu, Y., Chen, Z.: SySeVR: a framework for using deep learning to detect software vulnerabilities. *IEEE Trans. Dependable Secure Comput.* **19**(4), pp. 2244–2258 (2022)
 51. Wen, X.-C., Chen, Y., Gao, C., Zhang, H., Zhang, J.M., Liao, Q.: Vulnerability detection with graph simplification and enhanced graph representation learning. In: *2023 IEEE/ACM 45th International Conference on Software Engineering (ICSE)*, Melbourne, Australia, pp. 2275–228 (2023)
 52. Zhang, J., Liu, Z., Hu, X., Xia, X., Li, S.: Vulnerability detection by learning from syntax-based execution paths of code. *IEEE Trans. Software Eng.* **49**(8), 4196–4212 (2023)
 53. Verdi, M., Sami, A., Akhondali, J., Khomh, F., Uddin, G., Motlagh, A.K.: An empirical study of C++ vulnerabilities in crowd-sourced code examples. *IEEE Trans. Softw. Eng.* **48**(5), pp. 1497–1514 (2022)
 54. Zhang, D., Ahmad, W.U., Tan, M., Ding, H., Nallapati, R., Roth, D., Ma, X., Xiang, B.: Code representation learning at scale. In: *The Twelfth International Conference on Learning Representations*. Vienna Austria, (2024)
 55. Cong, B.V., Xuan, C.D.: A new framework for software vulnerability detection based on an advanced computing. *Comput. Mater. Continua* **79**(3), 3699–3723 (2024). <https://doi.org/10.32604/cmc.2024.050019>
 56. Do Xuan, C., Mai, D.H., Thanh, M.C., et al.: A novel approach for software vulnerability detection based on intelligent cognitive computing. *J. Supercomput.* **79**, 17042–17078 (2023). <https://doi.org/10.1007/s11227-023-05282-4>
 57. Xuan, C.D.: A new approach to software vulnerability detection based on CPG analysis. *Cogent Eng.*, 10(1) (2023). <https://doi.org/10.1080/23311916.2023.2221962>
 58. Do, C.X., Luu, N.T., Nguyen, P.T.L.: Optimizing software vulnerability detection using RoBERTa and machine learning. *Autom. Softw. Eng.* **31**, 40 (2024). <https://doi.org/10.1007/s10515-024-00440-1>
 59. Cho, D.X., Son, V.N., Duc, D.: Automatically detect software security vulnerabilities based on natural language processing techniques and machine learning algorithms. *J. ICT Res. Appl.* **16**(1), 70–87 (2022). <https://doi.org/10.5614/itbj.ict.res.appl.2022.16.1.5>
 60. Bui, V.-C., Do, X.-C.: Detecting software vulnerabilities based on source code analysis using GCN transformer. In: *2023 RIVF International Conference on Computing and Communication Technologies (RIVF)*, Hanoi, Vietnam, pp. 112–117 (2023). <https://doi.org/10.1109/RIVF60135.2023.10471834>

Publisher's Note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.

Springer Nature or its licensor (e.g. a society or other partner) holds exclusive rights to this article under a publishing agreement with the author(s) or other rightsholder(s); author self-archiving of the accepted manuscript version of this article is solely governed by the terms of such publishing agreement and applicable law.